



Chapter 12

Project 3.8: Integrated Data Acquisition Web Service

In many enterprise applications, data is provided to several consumers. One way to do this is to define an API that provides data (and the meta-data) for subsequent use. In this chapter, we guide you through the transformation of Project 2.5 schema information into a larger OpenAPI specification. We will also build a small Flask application that provides the core acquire-cleanse-convert process as a web service.

We'll cover a number of skills in the chapter:

- Creating an OpenAPI specification for a service to acquire and download data
- Writing a web service application to implement the OpenAPI specification
- Using a processing pool to delegate long-running background tasks

This is a bit of a deviation from a straight path of acquiring and cleaning data. In some enterprises, this deviation is needed to publish useful data to a wider audience.

We'll begin with a description of the behavior of this RESTful API server.

12.1 Description

In ***Chapter 8, Project 2.5: Schema and Metadata***, we used **Pydantic** to generate a schema for the analysis data model. This schema provides a formal, language-independent definition of the available data. This can then be shared widely to describe the data and resolve questions or ambiguities about the data, the processing provenance, the meaning of coded values, internal relationships, and other topics.

This specification for the schema can be extended to create a complete specification for a RESTful API that provides the data that meets the schema. The purpose of this API is to allow multiple users — via the `requests` module — to query the API for the analytical data as well as the results of the analysis. This can help users to avoid working with out-of-date data. An organization creates large JupyterLab servers to facilitate doing analysis processing on machines far larger than an ordinary laptop.

Further, an API provides a handy wrapper around the entire acquire-and-clean process. When a user requests data for the first time, the processing steps can be started and the results cached. Each subsequent request can download available data from a filesystem cache, providing rapid access. In the case of a failure, the logs can be provided as an alternative to the final data.

We won't dive deeply into REST design concepts. For more information on RESTful design, see <https://hub.packtpub.com/creating-restful-api/>.

Generally, a RESTful API defines a number of paths to resources. A given path can be accessed by a number of methods, some of which will get the resource. Other methods may post, patch, put, or delete the resource. The defined HTTP methods offer handy mapping to the common **Create-Retrieve-Update-Delete (CRUD)** conceptual operations.

Here are the common cases:

- A path without a final identifier, for example, `/series/`. There are two common cases here:
 - The `GET` method will retrieve the list of available resources of the given type.
 - The `POST` method can be used to create a new instance of this type. This is the conceptual “Create” operation.
- A path with an identifier. For example, `/series/Series_4`. This is a specific resource. There are several methods that might be implemented:
 - The `GET` method will retrieve the resource. This is the “Retrieve” conceptual operation.
 - The `PUT` and `PATCH` methods can be used to replace or update the resource. These are two forms of the conceptual “Update” operation.
 - The `DELETE` method can be used to remove the resource. This is the “Delete” conceptual operation.

It becomes imperative to consider a RESTful web service as a collection of resources. Talking about resources can make it difficult to talk about a RESTful request that initiates processing. It raises the question of what resource describes an activity such as processing samples. We’ll start by considering the data series as the most important resource provided by this service.

12.1.1 The data series resources

The primary resource for this API is the data series. As shown in the previous section, *[OpenAPI 3 specification](#)*, a path with `/2023.02/series/<id>` can be used to extract the data for a named series. The 2023.02 prefix allows the API to evolve to a newer version while leaving older paths in place for compatibility purposes.

The use of **semantic versioning (semver)** is common, and many APIs have something like “v1” in the path. Yet another alternative is to include the version information in the `Accept` header. This means the

URIs never change, but the schema for the response can change based on the version information provided in the header.

The various “series” routes provide direct access to the data resources. This seems appropriate since this is the primary purpose of the service.

There is an additional class of resources that might be of interest: the background processing used to create the data. As noted above, projects like ***Chapter 11, Project 3.7: Interim Data Persistence***, are the essential foundation for processing done by this RESTful API. The acquire and clean applications can be run in the background to create data for download.

A focus on resources is essential for making useful RESTful APIs.

Even when describing processing or state changes, the focus must be on the resource that undergoes the state change.

The methods available in HTTP (GET, POST, PUT, PATCH, and DELETE, for example) are effectively the verbs of the API’s language. The resources are nouns.

12.1.2 Creating data for download

The primary purpose of the RESTful API is to store and download clean data for analysis work. This can be a relatively straightforward application that offers data files from a well-known directory. The work involves matching RESTful requests against available files, and returning appropriate status codes when requests are made for files that don’t exist.

A secondary purpose is to automate the creation of the data for download. The RESTful API can be a wrapper around the complete acquire, clean, and persist pipeline. To do this, the API will have two distinct kinds of requests:

- Requests to download existing, cached data. The resource type is clear here.
- Requests to start the creation of new data; this will lead to cached data available for download. The resource type for processing isn't as clear.

An operation or action does have some static resources that can be used with a RESTful API. Here are two common resource types for activities:

- A "current status" resource that reflects the work being done right now
- A "processing history" resource that reflects work completed: this is often the log file for the acquisition processing

The control of processing by a RESTful API can work by creating and examining processing status or history as a distinct resource type:

- A path with a POST request will start an asynchronous, background process. This will also create a new processing history resource. The response body provides a transaction identifier referring to this new processing history.
- A path with a transaction identifier and a GET request will return the background processing details; this should include the current or final status as well as the log.

For sophisticated frontend processing, a web socket can be created to receive ongoing status reports from the background process. For a less sophisticated frontend, polling every few seconds can be done to see whether the processing has finished and the data is available for download.

With both processing history resources and data resources, the following two sets of paths are necessary:

- `/series/<id>` paths that refer to specific series, already available in the cache. These resources are accessed exclusively with the GET

method to download data.

- `/creation/<id>` paths that refer to background processing jobs to create a new series of data. These resources will use the POST method to start a background job, and the GET method to check the status of a job.

This set of paths (and the associated methods) allows a user to control processing and check the results of processing. The user can ask for available datasets and download a specific dataset for analysis.

12.2 Overall approach

We'll take some guidance from the C4 model (<https://c4model.com>) when looking at our approach.

- **Context** For this project, the context diagram has several use cases: listing available data, downloading available data, starting a process to acquire data, and checking the status of a process acquiring data.
- **Containers** Ideally, this runs on a single container that hosts the web service as well as the processing. In some cases, multiple containers will be required because the processing demands are so huge.
- **Components** There are two significantly different collections of software components: the web service, and the application programs that run in the background to acquire and clean the data.
- **Code** The acquiring and cleaning applications have already been described as separate projects. We'll focus on the web service.

We'll decompose the web service application into several components. The following diagram shows the relationship between the RESTful API service and the applications that are run to acquire and clean data.

The component diagram is shown in [*Figure 12.1*](#).

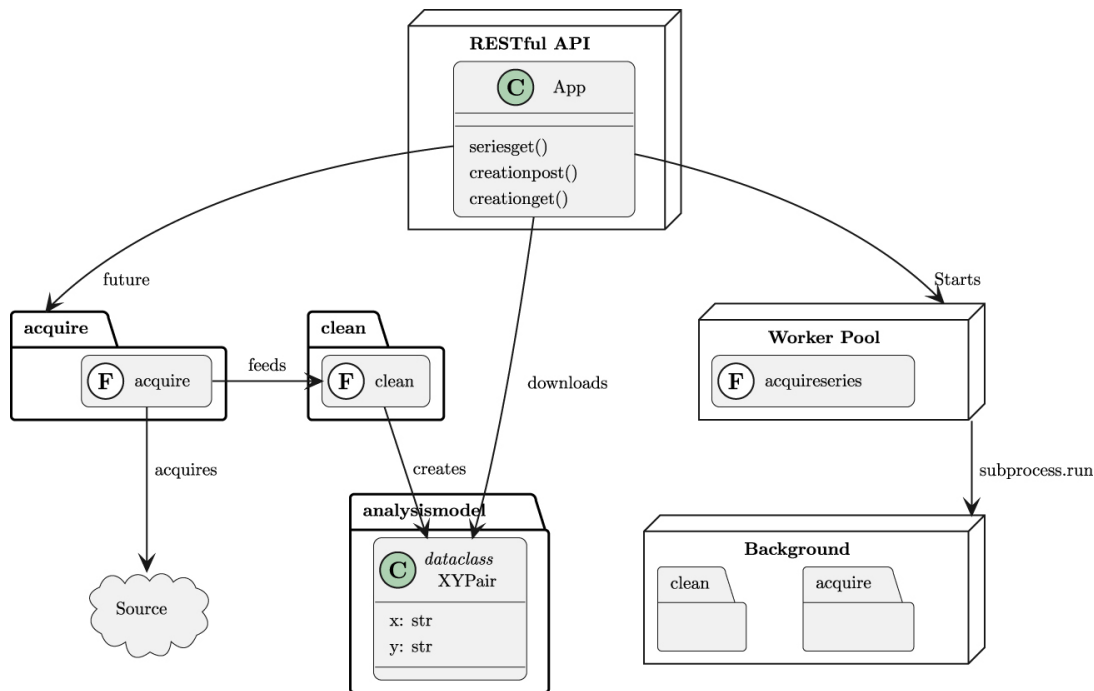


Figure 12.1: Application components

This diagram shows three separate processes:

- The **RESTful API** process that handles HTTP requests from clients.
- The **Worker Pool** collection of processes that are managed by the `concurrent.futures` module. Each worker will be running a single function, shown as `acquire_series`, that's defined in the same module as the **RESTful API** service.
- The **Background** process that is executed by a worker in the worker pool. This uses the `subprocess` module to run an existing CLI application.

When the API service starts, it uses `concurrent.futures` to create a pool of workers. A request to acquire and clean data will use the `submit()` method of the pool to create a **future**. This future is a reference to a subprocess that will — eventually — return the final status of the acquire and clean job. The subprocess that implements the future will evaluate the `acquire_series()` function defined in the same module as the RESTful API application to do the work.

When the `acquire_series()` function finishes the processing, it will have created a file that can be downloaded. Via the future object, it will

also provide some status information to the RESTful API service to indicate the processing is done.

One suggested implementation for the `acquire_series()` function is to use `subprocess.run()` to execute the `acquire` and `clean` applications to gather and cleanse source data. There are some other choices available. The most important alternative is to import these two other modules, and execute them directly, rather than creating a subprocess. This direct execution has the advantage of being slightly faster than spawning a subprocess. It has the disadvantage of making it more complicated to create a separate log file each time the **acquire** and **clean** application is executed.

We'll take a look at the OpenAPI specification for the RESTful API first. This helps to characterize the overall UX.

12.2.1 OpenAPI 3 specification

A RESTful API requires a clear description of the requests and responses. The OpenAPI specification is a formal definition of RESTful web services. See <https://www.openapis.org>. This document has a version identifier and some information about the service as a whole. For this project, the most important part is the **paths** section, which lists the various resource types and the paths used to locate those resources. The **components** section provides the needed schema definitions.

An OpenAPI document often has an outline like this:

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "The name of this service",
    "description": "Some details.",
    "version": "2023.02"
  }
  "paths": {
    "..."
```



```
    }  
    "components": {  
        "parameters": {"..."},  
        "schemas": {"..."}  
    }  
}
```

The details of the paths and components have been elided from this overview. (We've used `"..."` in place of the details.) The idea is to show the general structure of an OpenAPI specification. While JSON is the underlying format commonly used for these specifications, it can be hard to read. For this reason, it's common to use YAML notation for OpenAPI specifications.

Think of the OpenAPI specification as a binding contract.

The acceptance test suite should be Gherkin scenarios with a very direct mapping to the OpenAPI specification.

For more on the idea of OpenAPI to Gherkin, see

<https://medium.com/capital-one-tech/spec-to-gherkin-to-code-902e346bb9aa>.

The OpenAPI paths define the resources made available by a RESTful API. In this case, the resources are cleaned files, ready for analysis.

We'll often see entries in the **paths** section that look like the following YAML snippet:

```
"/2023.02/series":  
  get:  
    responses:  
      "200":  
        description: All of the available data series.  
        content:  
          application/json:  
            schema:  
              $ref: "#/components/schemas/series_list"
```

This shows a path that starts with an API version number (in this example, calendar versioning, “calver”, is used) and a resource-type, `series`. Any given path can be accessed by a variety of methods; in this example, only the `get` method is defined.

One kind of response is defined for requests to this path and method combination. The response will have a status code of 200, meaning normal, successful completion. The description is there to explain what this resource will be. A response can define a variety of content types; in this example, only `application/json` is defined. The schema for this is provided elsewhere in the OpenAPI specification, in the `components/schemas` section of the document.

The use of a `$ref` tag within the specification permits common definitions, such as schemas and parameters, to be collected under the `components` section, permitting reuse. This follows the **DRY (Don’t Repeat Yourself)** principle of software design.

It can be difficult to get the syntax correct in an OpenAPI specification. It’s helpful to have an editor that validates the specification. For example, <https://editor.swagger.io> provides an editor that helps confirm the specification is internally consistent. For readers using tools such as JetBrains’ PyCharm, there’s a plug-in editor:

<https://plugins.jetbrains.com/plugin/14837-openapi-swagger-editor>.

When a path has an identifier in it, then this is shown with the path name of the form `"/2023.02/series/<series_id>"`. The `<series_id>` is defined in the `parameters` section of this request. Since parameters are sometimes reused, it’s helpful to have a reference to a component with the common definition.

The whole request might start like this:

```
"/2023.02/series/<series_id>":  
  get:  
    description:
```

```
Get series data as text ND JSON.  
parameters:  
- $ref:  
  "#/components/parameters/series_id"  
responses:  
...
```

The details of the **responses** section have been omitted from this example. The parameter definition — in the `components` section — might look like this:

```
series_id:  
  name: series_id  
  in: path  
  required: true  
  description: Series name.  
  schema:  
    type: string
```

This provides a wealth of details about the `series_id` parameter, including the description and a formal schema definition. For simple APIs, the name of the parameter and the reference label under `components` are often the same. In more complex cases, a parameter name might be reused, but have distinct semantics in distinct contexts. A generic word such as `id` might be used in several different paths, leading to the reference label being something more descriptive than `id`.

The content for ND JSON is considered an extension to standard MIME types. Therefore the content definition for a response that includes data might look like this:

```
content:  
  application/x-ndjson:  
    schema:  
      $ref: "#/components/schemas/samples"
```

The schema is a challenge because it pushes the boundaries of what JSON Schema can describe. It looks as follows:

```
samples:
  description: >
    Acquired data for a series in ND JSON format.
    See http://ndjson.org and https://jsonlines.org.
  type: string
  format: "(\\{.*?\\}\\n)+"
```

The format information describes the physical organization of ND JSON data, but doesn't provide any details on the structure of the schema for each individual row. The additional schema details can either be added to the description, or a separate label, distinct from other JSON schema labels, can be used, for example, "ndjson-schema:".

12.2.2 RESTful API to be queried from a notebook

The RESTful API service must be a wrapper around application programming that can perform the required processing. The idea is to put as little processing as possible into the RESTful API. It serves as a very thin — almost transparent — interface to the “real work” of the application. For this reason, projects such as [*Chapter 11, Project 3.7: Interim Data Persistence*](#) are the essential foundation of this RESTful API.

As noted in [*Figure 12.1*](#), the **Background** processing is completely outside the RESTful API. This separation of concerns is absolutely essential. The general processing of samples can be performed with a CLI or through the RESTful API and create identical results.

If additional processing — for example, additional cleaning — is done by the RESTful service, then there are results that can't be reproduced from the CLI. This means the acceptance test suites have distinct results. This will lead to problems when a change is made to the underlying **acquire** or **clean** application and the “extra” processing that was jammed into the RESTful service now appears to be broken.

A common source of problems in enterprise software is the failure to honor the **Interface Segregation** design principle. A complex application may be supported by several collaborating organizations. When

one organization is slow to respond to requests for changes, another organization may step in and make a bad design decision, implementing processing in the API interface that should have been part of a background module with a proper CLI interface. The urge to be responsive to customers can often overshadow the importance of the separation of concerns.

For this project, the server can be built as a single process, avoiding the need for the distributed cache. Further, because the data series and the processing logs are all simple files, a database is not required; the local filesystem is perfectly suited to this service.

To create a more scalable solution, a library such as **celery** can be used to create a more robust distributed worker pool. This isn't needed for a small server, however.

In the next section, we'll review how processing can be started by a RESTful API.

12.2.3 A POST request starts processing

The general approach to creating a new resource is to make a **POST** request to a path. This will either return a 400 error status or it will issue a redirect (301) to a new path to retrieve the status of the background processing. This pattern is called the **Post-Redirect-Get** design pattern. It permits a user interacting with a browser to use the **back** button to perform the **GET** method again; it prevents the **back** button from submitting a duplicate request.

For a client application making a request via **requests** the redirect is essentially invisible. The request history will reveal the redirection. Also, the full URL recorded in the response will reflect the redirection.

The general processing for this route, then, is as follows:

1. Validate all of the parameters to make sure they describe the series and the source of the data. If there is anything amiss, a JSON response with the details of the problem must be returned, with a status code of 400 to indicate the request is invalid and must be changed.
2. Use the worker pool `submit()` method to create a `Future` object. This object can be saved in a local cache by the RESTful API. This cache of `Future` objects can be queried to see the background processing currently being performed. The future's result is usually something indicative of success or failure; for example, the return code from the subprocess – usually a zero indicates success.
3. Use the `redirect()` function in the Bottle framework to return the status code to direct a client to another URL for the status of the just-created `Future` object. Separately, a GET request will prepare a JSON document with the status of the job creating the data.

When using a framework like Bottle, this function is marked with a `@post("/2023.02/creation")` decorator. This names the POST method and the path that will be handled by the function.

The log files from processing can be the longer-term repository of processing history. The GET request for status will return a log and possibly the state of an active `Future` object. We'll look at this request next.

12.2.4 The GET request for processing status

The initial POST request to start processing will redirect to a GET request that reveals the status of the processing. The initial response may have almost no other details beyond the fact that the processing job has started.

This status path should return one of two things:

- A 404 status if the process ID is unknown. This would mean no previous request had been made with this identifier and no current request has this identifier, either.

- A 200 status with JSON content that includes some combination of two things: the state of a future object and the log file.

Most users will only care about the state of the `Future` object. In the case of developers, however, who are adding features to data acquire or data cleaning applications, then the log might be important support for observability.

When using a framework like Bottle, this function is marked with a `@get("/2023.02/creation/<job_id>")` decorator. This provides the method and the path that will be handled by the function. The use of `<job_id>` parses this section of the path and provides the value as a separate parameter to the function that implements this route.

Once the processing is complete, a subsequent request can provide the data. We'll look at this next.

12.2.5 The GET request for the results

This path should return one of two things:

- A 404 status if the series identifier is unknown.
- A 200 status with the ND JSON content. This has a MIME type of `application/x-ndjson` to indicate it's an extension to the standard collection of MIME types.

When using a framework like Bottle, this function is marked with a `@get("/2023.02/series/<series_id>")` decorator. The use of `<series_id>` parses this section of the path and provides the value as a separate parameter to the function that implements this route.

A more sophisticated implementation can check for an `Accept` header in the request. This header will state the preferred MIME type, and might have `text/csv` instead of `application/x-ndjson`. The use of this header permits a client to make requests for data in a format the application finds most useful.

12.2.6 Security considerations

A RESTful API requires some care to be sure the requests fit with the overall enterprise information access policies. In some cases, this might mean individual access controls to be sure each person can access permitted data. There are numerous **Single Sign-On (SSO)** products that can handle the identity of individuals.

Another common approach is to have an API work with assigned API keys. The team supporting the API can provide unique API key values to known users or teams. Within most enterprises, there's little need for automating the assignment of API keys for internal-facing APIs. The set of valid API keys may be reduced or expanded to reflect organizational merges and splits.

API key values are sent from the client to the server to authenticate the user making a request. They are never sent from the server to a client. The API keys can be kept in a simple text file; the file's permissions should restrict it to read-only access by the account handling the service as a whole. This requires administrators to take steps to manage the file of API keys to avoid damaging it or revealing it to unauthorized users.

When working with API keys, there are a number of ways the client can provide the key with each API request. One of the more popular techniques is to use these complementary security features:

- The HTTPS protocol, where all of the communication between client and server application is encrypted.
- The HTTP **Authorization** header with **Basic** authorization. This header will have a username and the API key as the password.

The use of the **Authorization** header is often very simple for a client tool. Many libraries — for example, the **requests** library — offer an object class that contains the username and API key. Using the `auth=` parameter on a request function will build the appropriate header.

The use of HTTPS includes **Transport Layer Security (TLS)** to keep the content of the **Authorization** header secret. The **requests** package handles this politely.

On the server side, each of these must be handled by our RESTful API application. Using HTTPS is best done by running the **Bottle** application inside another server. We could, for example, create an NGINX and uWSGI configuration that would run our RESTful app inside a containing server. Another choice is to use a Python-based server such as Paste or Gunicorn to contain the **Bottle** application. It's essential to have a container server to handle the details of HTTPS negotiation.

Processing the **Authorization** header is something best done within the RESTful API. Some routes (i.e., the `openapi.yaml`) should not include any security considerations. Other routes — specifically those that cause state changes — may be limited to a subset of all users.

This suggests the list of users includes some permissions as well as their API key. Each route needs to confirm the **Authorization** header has a known user and the correct key. The `request.auth` property of the `request` object is a two-tuple with the username and API key value. This can be used to decide whether the request is generally acceptable, and also to decide whether a state-changing **Post** operation is permitted for the given user. This kind of processing is often implemented as a decorator.

We won't dig deeply into the design of this decorator. For this project, with so few resources, a repeated `if` statement inside each function is acceptable.

12.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder
- Acceptance tests in the `tests/features` and `tests/steps` folders

- Unit tests for the application modules in the `tests` folder
- An application for the RESTful API processing

We'll start by looking at the acceptance test cases, first. They'll be rather complex because we need to start the RESTful API service before we can access it with a client request.

12.3.1 Acceptance test cases

Back in *[Chapter 4, Data Acquisition Features: Web APIs and Scraping](#)*, specifically *[Acceptance tests using a SQLite database](#)*, we looked at ways to describe a scenario that involved a database service.

For this project, we'll need to write scenarios that will lead to step definitions that start the RESTful API service.

There's an important question about setting the state of the RESTful API server. One approach to setting a state is by making a sequence of requests as part of the scenario. This is often appropriate for this application.

If the server's state is reflected in the file system, then seeding proper files can be a way to control the state of the API server. Rather than run an acquire and clean process, a test scenario can inject the appropriate status and log files into a working directory.

Some developers have a feeling that a database (or a distributed cache) is required for RESTful APIs. In practice, it's often the case that a shared file system is sufficient.

Using files is not uncommon in practice. A database to share state is not **always** required for RESTful APIs.

Using the file system for the state makes acceptance testing work out nicely. The proper files can be created to initialize the service in the state described by the given steps in the test scenario.

A complicated scenario could look like the following:

```
@fixture.REST_server
```

```
Scenario: Service starts and finishes acquiring data.
```

```
    Given initial request is made with path "/api/2023.02/creation" and
        method "post" and
        body with {"series": "2", "source": "Anscombe_quartet_data.csv"}
    And initial response has status "200", content-type "application/json"
    And initial response has job-id
    When polling every 2 seconds with path "/api/2023.02/creation/job-id" and
        method "get" finally has response body with status "Done"
    Then response content-type is "application/json"
    And response body has log with more than 0 lines
    And response body has series "Series_2"
    And response body has status "done"
```

For more background on creating a fixture, see [*Acceptance tests*](#) in [*Chapter 4, Data Acquisition Features: Web APIs and Scraping*](#). This scenario references a fixture named `REST_server`. This means the `environment.py` must define this fixture, and provide a `before_tag()` function that will make sure the fixture is used.

The given steps specify an initial query and response. This should set the required state in the API server. This request for processing will initiate the acquire and clean processing. The `When` step specifies a sequence of actions that include polling periodically until the requested processing finishes.

Note the path provided in the `When` statement. The text `job-id` is in the scenario's path. The step definition function must replace this template string with the actual job identifier. This identifier will be in response to the initial request in the given step. The `Given` step's definition function must save the value in the context for use in later steps.

The `Then` step confirms that series data was returned. This example does not show a very complete check of the result. You are encouraged

to expand on this kind of acceptance test scenario to be more complete in checking the actual results match the expected results.

For some applications, the retrieval of a tiny test case dataset may be a feature that helps test the application. The ordinary datasets the users want may be quite large, but a special, exceptionally small dataset may also be made available to confirm all the parts are working in concert.

A self-test resource is often essential for health checks, diagnostics, and general site reliability.

Network load balancers often need to probe a server to be sure it's capable of handling requests. A self-test URI can be helpful for this purpose.

A very subtle issue arises when trying to stop this service. It contains a worker pool, and the parent process needs to use the Linux `wait()` to properly terminate the children.

One reliable way to do this is to use `server.send_signal(signal.SIGINT)` in the function that starts the service to create the fixture for a scenario. This means the fixture function will have the following outline:

```
@fixture
def rest_server(context: Context) -> Iterator[Any]:
    # Create log file, base URI (code omitted)

    server = subprocess.Popen([sys.executable, "src/service.py"],
                              shell=False, stdout=context.log_file, stderr=subprocess.STDOUT)
    time.sleep(0.5) # 500 ms delay to allow the service to open a socket

    yield server # Scenario can now proceed.

    # 100 ms delay to let server's workers become idle.
    time.sleep(0.10)
    server.send_signal(signal.SIGINT)
```

```
# 100 ms delay to let API's subprocesses all terminate.  
time.sleep(0.10)
```

The various `sleep()` timings are generous over-estimations of the time required for the server subprocess to complete the various startup and shut-down tasks. In some cases, the OS scheduler will handle this gracefully. In other cases, however, disconnected child processes can be left in the list of running processes. These “zombie processes” need to be terminated manually, something we’d like to avoid.

On most Linux-derived OSs, the `ps -ef` command will show all processes. The `ps -ef | grep python` pipeline will show all Python processes.

From this list, any zombie worker pool processes should be apparent.

`signal.SIGINT` is the control-C interrupt signal. The Python process makes this an exception that will not be handled. When this exception exits from the `with` statement that created the process pool, a complete clean-up will be finished and no zombie processes will be left running.

Now that we’ve looked at the acceptance test that defines proper behavior, we can look at the RESTful API server application.

12.3.2 RESTful API app

The RESTful API application can be built with any of the available frameworks. Since a previous chapter ([*Chapter 4, Data Acquisition Features: Web APIs and Scraping*](#)) used the Bottle framework, you can continue with this small framework. Because Bottle is very much like Flask, when additional features are needed, the upgrade to Flask isn’t horribly complicated.

One of the advantages of using Flask for this application is an integrated client for writing unit test cases. The Bottle project can do everything that’s required, but it lacks a test client. When looking at unit testing, we’ll also look at unit test tools for the Bottle framework.

In ***OpenAPI 3 specification*** we looked at the OpenAPI specification for a specific path. Here's how that specification can be implemented:

```
from bottle import response, get

@get('/api/2023.02/series')
def series_list():
    series_metadata = [
        {"name": series.stem, "elements": series_size(series)}
        for series in DATA_PATH.glob("*.ndj")
    ]
    response.status = 200
    response.body = json.dumps(series_metadata, indent=2)
    response.content_type = "application/json"
    return response
```

This function builds a sequence of metadata dictionaries. Each item has a series name, which is used in a separate request to get the data. The size is computed by a small function to read the series and find the number of samples.

The `response` object is not always manipulated as shown in this example. This is an extreme case, where the value to be returned is not a Python dictionary. If the return value is a dictionary, the Bottle framework will convert it to JSON, and the content type will be set to `application/json` automatically. In this case, the result is a list of dictionaries; the Bottle framework will not automatically serialize the object in JSON notation.

An important part of the design is a cache to retain `Future` objects until the processing completes, and the data is available. One way to handle this is with a dataclass that keeps the parameters of the request, the `Future` object that will produce the results, and the assigned job identifier.

This structure for each `Future` object might look like the following example:

```
from concurrent import futures
from dataclasses import dataclass, field
from pathlib import Path
import secrets

@dataclass
class AcquireJob:
    series: str
    source_path: Path
    output_path: Path
    future: futures.Future = field(init=False)
    job_id: str = field(default_factory=lambda:
        \secrets.token_urlsafe(nbytes=12))
```

This keeps the parameters for the request as well as the processing details. The values for `series`, `source_path`, and `output_path` are built from the parameters provided when making an initial request. The paths are built from supplied names and include the base path for the working directory the server is using. In this example, the user's input is limited to the series name and the data source. These come from a small domain of valid values, making it relatively easy to validate these values.

The RESTful API can then create the output path within the appropriate directory of cleaned data.

The value for the `job_id` attribute is computed automatically when an instance of the `AcquireJob` class is created.

The value for the `future` attribute is set when the `submit()` method is used to submit a processing request to process a pool of waiting workers.

The worker pool needs to be created before any work can be done by the RESTful API. The startup can look like the following:

```
from concurrent import futures
import urllib.parse
```

```
WORKERS: futures.ProcessPoolExecutor

# Definitions of all of the routes

if __name__ == "__main__":
    # Defaults...
    acquire_uri = "http://localhost:8080"
    # Parse a configuration, here; possibly overriding defaults
    uri = urllib.parse.urlparse(acquire_uri)
    with futures.ProcessPoolExecutor() as WORKERS:
        run(host=uri.hostname, port=uri.port)
```

Each route is handled by a separate function. Because of this, the Bottle (as well as the Flask) framework expects the worker pool to be a global object shared by all of the route-handling functions. In the event of a multi-threaded server, a lock must be used before a write access to the `WORKERS` global.

Similarly, the cache of `AcquireJob` instances is also expected to be a global object. This is updated only by the route-handling function to handle initiating a processing request. This cache will be queried by a route that shows the status of a processing request. In the event of a multi-threaded server, a lock must be used before adding a new item to the global cache of working jobs.

In some cases, where the load is particularly heavy, thread-local storage may be needed for any processing done by the various functions in the RESTful API implementation. The `request` and `response` objects, in particular, are already in thread-local storage. Ideally, there is very little processing done by these functions, minimizing the number of objects that need to be created and kept in an instance of `threading.local`.

There are a few special considerations for the unit tests for this project. We'll look at those in the next section.

12.3.3 Unit test cases

Some frameworks — like **Flask** — offer a test client that can be used to exercise an application without the overheads of starting a server and a worker pool.

The **Bottle** framework doesn't offer a test client. An associated project, **boddle**, offers a way to build a mock `request` object to support unit testing. See <https://github.com/keredson/boddle>.

The **WebTest** project is an alternative for writing unit tests. A **WebTest** fixture contains the Bottle application and provides requests and responses through the internal WSGI interface. This avoids the need to start a complete server. It also permits some monkey-patching of the Bottle application to mock components. See <https://docs.pylonsproject.org/projects/webtest/en/latest/>.

It seems best to use the very sophisticated `WebTest` client that's part of the **Pylons** framework. This client can execute the unit tests.

It's sometimes helpful to note that functions with decorators are composite objects. This means the “unit” test isn't testing the decoration and the function in isolation from each other. This lack of separate testing can sometimes lead to difficulty in debugging the root cause of a test case failure. A problem may be in the function, it may be the `@route` decorator, or it may be any authorization decorator that's also part of the composite function being tested.

It seems easier to test the composite route functions, using appropriate log messages for debugging. While this doesn't follow the strict idea of testing each component in isolation, it does work well for testing each route with appropriate mocks. For example, we can mock the worker pool, avoiding the overhead of starting a subprocess when testing.

Here's an example of a test function using **WebTest** to exercise a **Bottle** route:

```
from unittest.mock import sentinel, Mock, call
from pytest import fixture, MonkeyPatch
from webtest import TestApp
import service

def test_test(monkeypatch: MonkeyPatch) -> None:
    monkeypatch.setitem(service.ACCESS, "unit-test", "unit-test")
    app = TestApp(service.app)
    app.authorization = (
        "Basic", ("unit-test", "unit-test")
    )
    response = app.get("/api/2023.02/test")
    assert response.status_code == 200
    assert response.json['status'] == "OK"
```

`service.app` is the global `app` object in the RESTful API application. This is an instance of the `Bottle` class. `service.ACCESS` is the global list of usernames and their expected API keys. This is monkey-patched by the test to force in a specific test username and test API Key. This initial setup is something that might be used by a number of tests and should be defined as a reusable fixture.

When the `app.get()` request is made, the test harness will execute the `route` function and collect the response for examination by the `test` method. This makes a direct function call, avoiding the overhead of a network request.

One of the reasons for choosing to use **Flask** instead of **Bottle** is the availability of a test client that can simplify some of this test setup.

12.4 Summary

This chapter integrated a number of application programs under the cover of a single RESTful API. To build a proper API, there were several important groups of skills:

- Creating an OpenAPI specification.

- Writing a web service application to implement the OpenAPI specification.
- Using a processing pool to delegate long-running background tasks. In this example, we used `concurrent.futures` to create a future promise of results, and then compute those results.

The number of processes involved can be quite daunting. In addition to the web service, there is a processing pool, with a number of sub-processes to do the work of acquiring and cleaning data.

In many cases, additional tools are built to monitor the API to be sure it's running properly. Further, it's also common to allocate dedicated servers to this work, and configure `supervisord` to start the overall service and ensure the service continues to run properly.

12.5 Extras

Here are some ideas for you to add to these projects.

12.5.1 Add filtering criteria to the POST request

The **POST** request that initiates acquire processing is quite complicated. See [***A POST request starts processing***](#) to see the processing it does.

We might name the function for this route `creation_job_post()` to make it clear that this creates jobs to acquire data in response to an HTTP POST request.

The list of tasks in this function includes the following:

1. Check the user's permissions.
2. Validate the parameters.
3. Build an `AcquireJob` instance with the parameters.
4. Update the `AcquireJob` instance with the `Future` object. The future will evaluate the `acquire_series()` function that does the work of acquiring and cleaning the data.

5. Return a JSON object with details of the submitted job, as well as headers and a status code to redirect to a request to get the job's status.

Some RESTful APIs will have even more complicated parameters. For example, users may want to filter the data to create a subset before downloading. This improves the UX by providing only the required data. It also allows analysts to share subsets of data without having to share the filtering code within the analyst community.

It can also improve the UX by performing filtering on larger, powerful servers. It can prevent having to download and filter data on a local laptop.

This is emphatically **not** a feature of the RESTful API. This must **first** be built as a feature of an application that reads and filters the clean data. This new application will create a new dataset, ready for download. The data set name might be a UUID, and an associated metadata file would contain the filter parameters.

The implementation requires the `creation_job_post()` function to now also validate the filter criteria. It must include the filter criteria in the `AcquireJob` instance that is built, and provide the filter criteria to the underlying `acquire_series()` function.

The `acquire_series()` function will have the most dramatic changes. It will run the acquire, clean, and filter applications as subprocesses. You may want to consider an integrated application that runs the other applications, simplifying the RESTful API.

This will, of course, lead to considerably more complicated acceptance test cases to be sure the data acquisition works with — and without — these additional filter criteria.

12.5.2 Split the OpenAPI specification into two parts to use \$REF for the output schema

The OpenAPI specification includes a number of schema. In [**OpenAPI 3 specification**](#), we showed a few key features of this specification.

It's not too difficult for an analyst to download the entire specification, and then locate the `components.schemas.seriesList` schema. This navigation through a JSON document doesn't involve too many challenges.

While this is not burdensome, some users might object. An analyst focused on a business problem should not be asked to also sort out the structure of the OpenAPI specification. An alternative is to decompose the specification into pieces and serve the pieces separately.

Specifically, the places where `"$ref"` references appear generally use a path of the form `#/components/schemas/...`. The path is a local URL, omitting the hostname information. This can be replaced with a full URL that refers to schema details on the RESTful API server.

We might use `http://localhost:8080/api/schemas/...` to refer to the various schema files stored as separate JSON documents. Each individual schema definition would have a distinct URI, permitting ready access to only the relevant schema, and ignoring other aspects of the OpenAPI specification.

This decomposes the OpenAPI specification into the overall specification for the service and separate specifications for a schema that describes downloadable datasets. It also requires adding a path to the RESTful API service to properly download the schema in addition to downloading the overall OpenAPI specification.

This leads to a few extra acceptance test cases to extract the schema as well as the overall OpenAPI specification.

12.5.3 Use Celery instead of `concurrent.futures`

The suggestion in ***Overall approach*** is to use the `concurrent.futures` module to handle the long-running data acquisition and cleaning processes. The API requests that initiate processing create a `Future` object that reflects the state of a separate subprocess doing the actual work. The RESTful API is free to respond to additional requests while the work is being completed.

Another popular package for implementing this kind of background processing is `celery`. See <https://docs.celeryq.dev/en/stable/getting-started/introduction.html>.

This is a bit more complicated than using the `concurrent.futures` module. It also scales elegantly to allow a large number of separate computers to comprise the pool of available workers. This can permit very large processing loads to be controlled by a relatively small RESTful API application.

Using Celery requires creating tasks, using the `@task` decorator. It also requires starting the worker pool separately. This means the overall RESTful API now has two steps to get started:

- The celery worker pool must be running.
- The RESTful API can then start. Once it's running, it can delegate work to workers in the pool.

For very large workloads, where the worker pool is spread across multiple computers, use of Celery's sophisticated management tools are required to be sure the pools are starting and stopping appropriately.

The core work of submitting work to the worker pool changes from `pool.submit()` to `celery_app.delay()`. This is a small programming change that permits using a more sophisticated and scalable worker pool.

There aren't any acceptance test changes for this. The features are identical.

The fixture definition required to start the RESTful API will be more complicated: it will have to start the **Celery** pool of workers before starting the RESTful API. It will also need to shut down both services.

12.5.4 Call external processing directly instead of running a subprocess

In ***Overall approach***, we suggested the work should be done by an `acquire_series()` function. This function would be evaluated by the `POOL.submit()` function. This would delegate the work to a worker, and return a `Future` object to track the state of completion.

In that section, we suggested the `acquire_series()` function could use `subprocess.run()` to execute the various components of the processing pipeline. It could run the `src/acquire.py` application, and then run the `src/clean.py` application, using the `subprocess` module.

This isn't the only way it could work. The alternative is to import these application modules, and evaluate their `main()` functions directly.

This means replacing the `subprocess.run()` function with the `acquire.main()` and `clean.main()` functions. This avoids a tiny overhead in Linux. It can be a conceptual simplification to see how the `acquire_series()` function creates the data using other Python modules.

This involves no changes to the acceptance test cases. It does involve some changes to the unit test cases. When using `subprocess.run()`, the unit test must monkey-patch the `subprocess` module with a mock that captures the argument values and returns a useful result. When replacing this processing with the `acquire.main()` and `clean.main()` functions, these two modules must be monkey patched with mocks that capture the argument values and return useful results.

